

DecoupleGS: Interactive 3D Gaussian Splatting for End-to-End Autonomous Driving Testing

Siying Li^{1,2}, Ying Ni^{1,2†}, Jie Sun^{1,2}, Jian Sun^{1,2}, and Haotian Shi^{1,2†}

¹ College of Transportation, Tongji University, Shanghai 201804, China

² Key Laboratory of Road and Traffic Engineering, Ministry of Education, Shanghai 201804, China

{siyingli,ying_ni,jie_sun,sunjian,shihaotian95}@tongji.edu.cn

Abstract. End-to-end (E2E) autonomous driving algorithms require rigorous closed-loop validation in simulation environments offering high visual fidelity, strong interactivity, and real-time performance. Existing approaches, from game engines to static neural rendering, inherently trade off these requirements and struggle with the dynamic scene composition essential for E2E testing. To bridge this gap, we propose a novel decoupled 3D Gaussian Splatting (3DGS) framework tailored for large-scale E2E evaluation. We fundamentally decompose scenes into a high-fidelity static background and manipulable dynamic agents using an object-centric canonical representation. To resolve resulting representational conflicts, we introduce three targeted modules: (1) asset compression via perceptual pruning and vector quantization for real-time traffic rendering; (2) map-guided geometric registration leveraging semantic topology to strictly align trajectories; and (3) proxy-based re-lighting transferring ambient illumination for seamless photometric integration. Extensive experiments demonstrate that DecoupleGS achieves a balanced fidelity-efficiency trade-off, improves metric and photometric consistency, and provides a practical closed-loop sensor simulation platform for E2E autonomous driving evaluation.

Keywords: Autonomous Driving · 3D Gaussian Splatting · End-to-End Testing · Sensor Simulation

1 Introduction

Autonomous driving (AD) is rapidly transitioning from classical modular pipelines to End-to-End (E2E) learning paradigms that map raw sensor observations directly to control commands [6, 8, 33]. While E2E algorithms reduce error accumulation [7, 16, 17], their safety and driving policies must be rigorously optimized across a broad, demanding set of scenarios. Since real-world data collection for such safety-critical scenarios is prohibitively costly and hazardous, high-fidelity virtual simulation has become an indispensable data engine for the continuous testing and optimization of E2E algorithms.

[†] Corresponding authors.

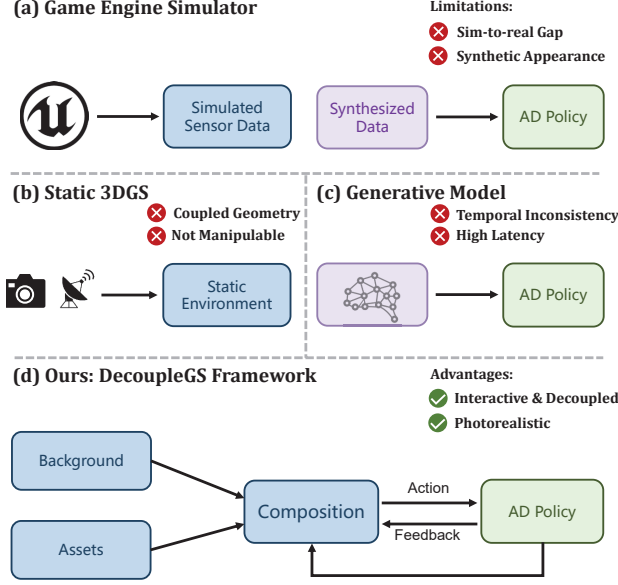


Fig. 1: Comparison of autonomous driving simulation paradigms. Traditional approaches face distinct limitations: (a) Game engine simulators suffer from sim-to-real gaps and synthetic appearances; (b) Generative models struggle with temporal inconsistencies and high inference latency; (c) Static 3DGS representations couple geometry and illumination, making dynamic manipulation impossible. In contrast, (d) our DecoupleGS framework introduces a modular composition of backgrounds and compressed assets, uniquely enabling an interactive, photorealistic, and closed-loop testing environment for AD policies.

For E2E testing, a practical closed-loop simulation platform must satisfy three key requirements: high visual and geometric fidelity, strong multi-agent interactivity, and sufficient real-time throughput [2, 9, 23, 31, 32]. However, existing sensor simulation approaches inherently trade off these requirements. Traditional game engines suffer from sim-to-real domain gaps, generative 2D models lack strict multi-view 3D consistency for spatial-temporal control loops, and static neural rendering methods lack the modularity to dynamically manipulate traffic flows.

Consequently, current methodologies cannot meet the integrated demands of a large-scale, rigorous testing pipeline. This gap stems from representational conflicts when seamlessly separating persistent static infrastructure from dynamic traffic participants. Merging these distinct entities introduces severe bottlenecks: storing and rendering multiple high-fidelity vehicles exhausts memory (efficiency conflict); unstructured neural coordinates fail to align with semantic metric maps (geometric conflict); and entangled environment lighting prevents dynamic assets from inheriting consistent illumination (photometric conflict).

To resolve these challenges, we propose DecoupleGS, a novel decoupled 3D Gaussian Splatting (3DGS) framework. The core task of this paper is to provide a comprehensive sensor simulation framework for E2E algorithms, achieving interactive, high-fidelity closed-loop evaluation that enables rigorous safety validation and dynamic multi-agent testing at scale. As compared in Fig. 1, while existing simulation paradigms suffer from the sim-to-real gap, temporal inconsistency, or coupled geometry, DecoupleGS uniquely bridges these gaps by organically composing persistent backgrounds and manipulable assets. We utilize an object-centric canonical representation to decompose the scene into a persistent high-fidelity static background and independent, compact 3DGS dynamic agents. Our pipeline integrates three targeted modules: an asset compression module utilizing perceptual redundancy pruning and vector quantization to mitigate memory bottlenecks; a map-guided geometric registration module leveraging semantic topology to strictly align agent trajectories; and a proxy-based relighting mechanism transferring ambient illumination for seamless photometric integration.

The main contributions of this study are threefold. First, we present a holistic 3DGS framework that integrates neural reconstruction fidelity with the interactivity and throughput required for large-scale closed-loop E2E testing. Second, we systematically address fundamental efficiency, geometry, and photometry conflicts by developing solver-agnostic modules for vehicle compression, map-guided registration, and relighting. Third, we validate that these modules enable plug-and-play insertion of multiple vehicles at interactive rates, establishing a robust and efficient data engine for E2E algorithms.

2 Related Work

2.1 Traditional Simulators and 2D Generative Methods

Game-engine-based simulators provide mature scene graphs, physically-based rendering primitives, and real-time rasterization pipelines, which yield strong interactivity and efficient runtime performance [2, 9, 23, 31, 32]. Their principal limitation for testing lies in the engineering cost of producing extensive collections of high-fidelity, diverse assets, and the reliance on hand-crafted materials and approximate physics, which often preserve a non-negligible visual or sensor-level domain gap from recorded real scenes. Conversely, generative 2D image synthesis—particularly latent diffusion models—has recently achieved impressive photorealism for scene synthesis and controllable simulation [15, 25, 30, 37, 41, 44], but these methods are fundamentally 2D. They do not provide an intrinsic, multi-view, physically consistent 3D representation and therefore struggle with high inference latency and temporal coherence under the high frame-rate demands of control loops.

2.2 Neural 3D Reconstruction and Rendering

By contrast, neural 3D reconstruction and neural rendering methods—most notably NeRF variants and the recent 3DGS paradigm—reconstruct geometry and

view-dependent appearance directly in a 3D coordinate frame, which reduces certain sim-to-real gaps and enables novel-view synthesis [1, 5, 12, 19, 27, 28, 43]. While recent frameworks advance closed-loop safety testing and dynamic scene composition [4, 14, 26, 42, 45, 47, 48], they still face bottlenecks for interactive E2E testing: many rely on pre-rendered alternatives, incorporate lighting into the reconstruction, or generally lack an efficient, map-aware modular decomposition that cleanly separates persistent infrastructure from movable traffic participants [13, 34, 36]. Consequently, naively reusing reconstructed vehicle instances across scenarios or moving them along arbitrary trajectories produces geometric and photometric inconsistencies.

2.3 Research Gaps for Interactive Neural Scene Composition

To address these challenges, recent attempts to make neural reconstructions interactive or to insert dynamic vehicles into neural scenes have advanced the state of the art [22, 29, 38, 46] but expose three concrete research gaps that directly impede closed-loop E2E evaluation.

Efficiency and Scalability. High-fidelity vehicle models derived from 3DGS or dense Gaussian representations typically contain a large number of primitives. Although 3DGS substantially accelerates rendering compared with early implicit NeRF-based methods, the composition of multiple detailed vehicle instances remains computationally demanding, often leading to severe frame-rate degradation when many agents co-exist in a scene. This limitation arises because most reconstruction-oriented pipelines prioritize offline visual fidelity and lack runtime-oriented compression or pruning mechanisms, which are essential for maintaining interactive performance in large-scale multi-agent simulation [42, 48].

Geometric Alignment and Metric Consistency. Neural reconstructions are typically represented in unstructured learned coordinates, whereas planning and control operate in metric map space, leading to alignment inconsistencies in closed-loop simulation. Existing object-centric methods [29, 46] focus on single-object editing or require per-scene optimization and therefore do not yet provide robust, map-aware registration for long-horizon multi-agent sequences.

Photometric Integration and Relighting. In neural scene representations, lighting is often entangled with geometry and appearance, preventing inserted vehicles from automatically inheriting consistent ambient illumination, shadows, and environmental reflections. Although recent relighting and compositing pipelines demonstrate promising capabilities for object-level relighting and shadow synthesis [49], they typically incur substantial computational overhead or assume a limited number of edited objects and fixed camera configurations. These constraints hinder their applicability to large-scale, interactive multi-vehicle simulation.

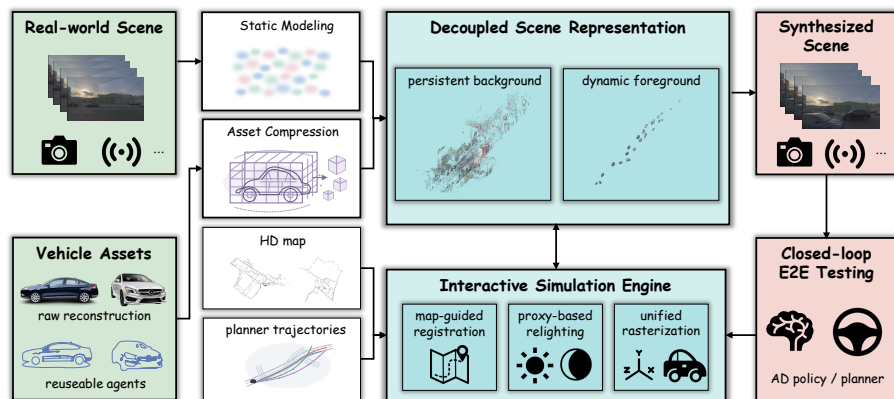


Fig. 2: Overview of the DecoupleGS framework. DecoupleGS decouples persistent static backgrounds from canonical dynamic assets, and integrates asset compression, map-guided registration, proxy-based relighting, and unified rasterization to support interactive closed-loop E2E evaluation.

3 Method

To enable large-scale closed-loop E2E evaluation, we aim to compose a photo-realistic yet interactive simulation where (i) the static infrastructure remains persistent across time, and (ii) a variable number of traffic agents can be instantiated, moved, relit, and rendered in response to planner-driven scene states. As illustrated in Fig. 2, DecoupleGS is organized around a decoupled neural scene representation and three tightly coupled functional modules. The static background and canonical dynamic assets are represented in separate coordinate spaces, while asset compression, map-guided registration, and proxy-based relighting respectively address the efficiency, geometric, and photometric conflicts that arise during dynamic scene composition. The transformed and relit primitives are finally composited through a unified rasterizer to produce multi-view observations for closed-loop E2E testing.

3.1 Decoupling and Unified Rendering

Applying 3DGS to interactive E2E testing is fundamentally bottlenecked by its inherent assumption of static scenes. We introduce a dual-stream architecture that represents the background and dynamic agents in separate coordinate spaces, mathematically fusing them prior to rasterization without the need for network inference.

Object-Centric Canonical Decomposition. Formally, the global scene Ω at timestamp t is defined as the spatial union of a time-invariant background field

$\mathcal{S}_{bg}$ and a set of K dynamic agents:

$$\Omega(t) = \mathcal{S}_{bg} \cup \left(\bigcup_{k=1}^K \mathcal{T}_k(t) \circ \mathcal{V}_k \right) \quad (1)$$

The background $\mathcal{S}_{bg}$ consists of Gaussian primitives anchored in the world coordinate system (WCS). Conversely, each dynamic agent k is parameterized as a canonical Gaussian volume $\mathcal{V}_k$ within a standardized local coordinate system (LCS). Given the agent’s 6-DoF rigid transform $\mathcal{T}_k(t) = [\mathbf{R}_k \mid \mathbf{t}_k] \in SE(3)$ mapping LCS to WCS, we apply the transformation to the spatial attributes (means μ and covariances Σ) of all primitives within $\mathcal{V}_k$:

$$\mu_i^{(W)} = \mathbf{R}_k \mu_i^{(L)} + \mathbf{t}_k, \quad \Sigma_i^{(W)} = \mathbf{R}_k \Sigma_i^{(L)} \mathbf{R}_k^\top \quad (2)$$

Crucially, 3DGS encodes view-dependent appearance via Spherical Harmonics (SH). When the geometry undergoes a rotation $\mathbf{R}_k$, the intrinsic SH coefficients $c_i^{(L)}$ must be correspondingly rotated to preserve correct view-dependent reflections relative to the new orientation. We apply the Wigner D-matrix $\mathbf{D}(\mathbf{R}_k)$ to rigorously align the SH bands:

$$c_i^{(W)} = \mathbf{D}(\mathbf{R}_k) c_i^{(L)} \quad (3)$$

While this mathematically preserves the asset’s intrinsic specularities under rigid motion, resolving the ultimate photometric mismatch with the new background illumination is deferred to our proxy-based relighting module (Sec. 3.4).

Unified Rasterization and Runtime Optimization. To ensure physically accurate occlusion, all primitives from $\Omega(t)$ are merged into a unified aggregate list $\mathcal{P}$. For each pixel u , primitives overlapping u are depth-sorted relative to the novel camera center and composited via continuous front-to-back α -blending. To maintain the robust interactive frame rates required for large-scale E2E testing, we integrate several hardware-level optimizations: (a) conservative frustum culling to aggressively bound $\mathcal{P}$; (b) decoupling the sorting radix, where the static background splats are cached and incrementally reused; and (c) computing batched spatial updates exclusively for the moving canonical volumes.

3.2 Asset Compression

Directly rendering high-dimensional raw 3DGS assets for dense traffic is computationally prohibitive. As detailed in Fig. 3, we introduce a semantic-aware compression pipeline consisting of importance scoring, pruning, and vector quantization to produce compact canonical assets suitable for real-time multi-agent composition.

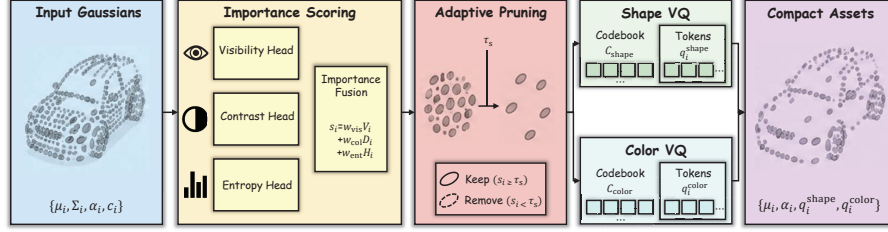


Fig. 3: Illustration of the semantic-aware asset compression module. Canonical vehicle Gaussians undergo explicit importance scoring (V_i, D_i, H_i), adaptive pruning via threshold τ_s , and high-dimensional attribute vector quantization to form compact canonical volumes.

Explicit Importance Scoring and Pruning. Instead of relying on black-box neural networks, we compute an explicit scalar importance score s_i for each primitive g_i that summarizes its visibility, photometric contribution, and local geometric saliency:

$$s_i = w_{\text{vis}} V_i + w_{\text{col}} D_i + w_{\text{ent}} H_i \quad (4)$$

Here, the visibility estimate V_i is the expected opacity contribution over sampled training views; the color contrast term D_i measures perceptual distinctiveness versus the local background; and the entropy term H_i captures texture complexity inside the primitive neighborhood. Primitives with s_i below a strict threshold τ_s (empirically set to 0.005) are pruned. This operation significantly reduces the primitive count, concentrating computational resources on essential structural details like wheels and chassis boundaries. The weighting coefficients w_{vis} , w_{col} , and w_{ent} are empirically determined via grid search to optimally balance geometric preservation and color fidelity, demonstrating robust generalization across different vehicle types.

Attribute Vector Quantization. To further minimize the memory footprint, Vector Quantization (VQ) is applied to the high-dimensional attributes of retained primitives. The covariance Σ_i and color coefficients c_i are replaced by their nearest entries in small, learnable codebooks $\mathcal{C}_{\text{shape}}$ and $\mathcal{C}_{\text{color}}$:

$$\Sigma_i \approx \mathcal{C}_{\text{shape}}[q_i^{\text{shape}}], \quad c_i \approx \mathcal{C}_{\text{color}}[q_i^{\text{color}}] \quad (5)$$

This discretization replaces heavy floating-point tensors with compact integer indices q_i . Specifically, we utilize attribute-specific codebooks (e.g., $K_{\text{color}} = 1024$ for SH color coefficients and $K_{\text{shape}} = 512$ for covariance attributes) optimized via Exponential Moving Average (EMA) during compression.

3.3 Map-Guided Geometric Registration

Integrating dynamic agents requires strict geometric alignment. It is crucial to note that for the ego-vehicle evaluated in closed-loop E2E testing, its trajectory

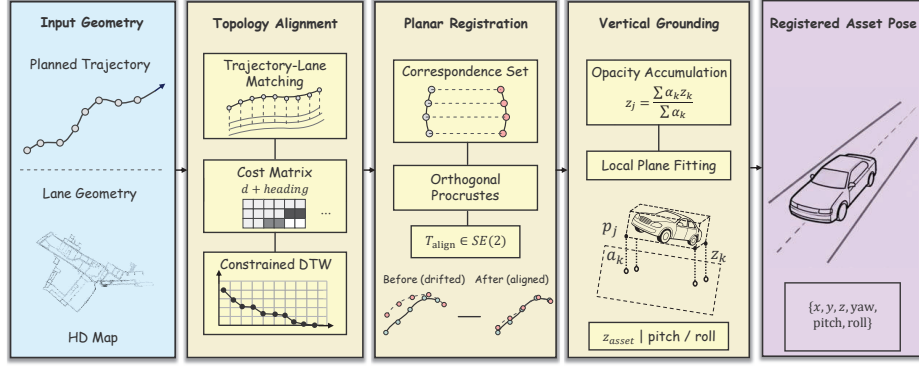


Fig. 4: The map-guided geometric registration pipeline. It enforces strict metric alignment via trajectory-to-lane DTW sequence alignment, $SE(2)$ Procrustes 2D transformation, and robust opacity-accumulated 3D vertical grounding.

is driven entirely by the unconstrained relative pose updates from the planner to preserve maneuvers like lane changes. The sequence-level registration described below, and illustrated in Fig. 4, is specifically designed to accurately inject background traffic agents into the reconstructed shared coordinate system.

Semantic Topology and Planar Alignment. We obtain vectorized lane centerlines $\mathcal{L}$ from dataset-provided HD maps when available, and use MapTRv2 [24] to extract lane topology otherwise. Given a background agent’s 2D trajectory $\mathcal{T} = \{p_t\}_{t=1}^T$ and the corresponding lane geometry, we build a cost matrix that penalizes both Euclidean distance and heading mismatch. We utilize constrained Dynamic Time Warping (DTW) to find the optimal warping path establishing trajectory-to-lane correspondences. Subsequently, an Orthogonal Procrustes analysis computes the globally optimal 2D rigid transform $T_{align} \in SE(2)$ to eliminate systemic lateral drift.

Vertical Grounding via Opacity Accumulation. Since 3DGS lacks a continuous surface mesh, standard ray-casting fails. We employ opacity-weighted depth accumulation to ground the vehicles. For anchor points p_j defined at the bottom corners of the asset’s bounding box, the vertical height z_j is computed as the opacity-weighted average of the intersected background gaussians within a vertical column $\mathcal{N}$:

$$z_j = \frac{\sum_{k \in \mathcal{N}} \alpha_k z_k}{\sum_{k \in \mathcal{N}} \alpha_k} \quad (6)$$

A local ground plane is then fitted using least-squares minimization over the anchors $\{z_j\}$. The final vertical position z_{asset} and attitude (pitch and roll) of the asset are derived from the estimated plane normal, ensuring that the wheels maintain physically plausible contact with the terrain.

3.4 Relighting

To resolve the photometric conflict where inserted assets retain intrinsic illumination that mismatches the novel environment, we develop a lightweight, proxy-based relighting module that operates without online neural network inference.

Local Probe Sampling and Linear SH Transfer. We treat the background SH coefficients as a dense field of light probes. For a vehicle located at x , we construct a local ambient descriptor $\mathcal{L}(x)$ by aggregating the SH coefficients of neighboring background gaussians c_g , weighted by their visibility V_g and spatial distance:

$$\mathcal{L}(x) = \frac{\sum_{g \in \mathcal{N}(x)} w_g c_g}{\sum_{g \in \mathcal{N}(x)} w_g}, \quad w_g = \exp\left(-\frac{\|x - \mu_g\|^2}{2\sigma_p^2}\right) V_g \quad (7)$$

To transfer this environmental lighting, we apply a pre-calibrated linear modulation operator to the canonical vehicle’s SH coefficients c_{can} :

$$c_{out} = \mathcal{T}(\mathcal{L}(x)) \otimes c_{can} + b(\mathcal{L}(x)) \quad (8)$$

where $\mathcal{T}(\cdot)$ and $b(\cdot)$ are efficient transformation matrices. Rather than relying on online neural network inference, these operators are derived offline by solving a least-squares regression over a synthetic dataset of canonical assets rendered under diverse pre-captured environmental probes. This analytical calibration completely avoids per-frame MLP inference while successfully rebaking the assets with the local lighting context.

Proxy-based Contact Shadows. To anchor the object visually, we synthesize contact shadows on the estimated local ground plane. We project a parametric shadow mask $M_{shadow}(u)$ scaled by the object footprint. Crucially, the intensity of the background gaussians under this mask is modulated dynamically by the dominant light intensity I_{dom} extracted from the DC component of the ambient descriptor $\mathcal{L}(x)$:

$$c_{final}(u) = c_{ground}(u) \cdot (1 - \lambda \cdot I_{dom} \cdot M_{shadow}(u)) \quad (9)$$

This efficient approximation ensures that shadows become appropriately faint in overcast conditions or intense under direct sunlight, completing the photometric integration seamlessly.

4 Experiments

This section evaluates DecoupleGS from four complementary perspectives: rendering efficiency, geometric and photometric fidelity, open-loop sim-to-real consistency, and closed-loop E2E testing. We further ablate the proposed compression, registration, and relighting modules to verify how each component resolves the efficiency, geometric, and photometric conflicts in dynamic neural scene composition.

Table 1: Asset compression results under different traffic densities. Best values are bolded, second-best values are shaded in gray, and DecoupleGS rows are highlighted. The final column summarizes whether a method consistently remains top-2 across all fidelity and efficiency metrics.

Size	Methods	Fidelity				Efficiency		Top-2 Consist.
		PSNR-Veh $\uparrow$	PSNR-All $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	FPS $\uparrow$	VRAM $\downarrow$	
S	Plenoxels [12]	21.45	23.06	0.795	0.510	8.5	2.5GB	No
	Vanilla 3DGS [19]	28.52	29.41	0.903	0.243	55.4	1.2GB	No
	LightGaussian [11]	26.21	27.50	0.865	0.281	75.2	180MB	No
	DecoupleGS (Ours)	28.10	29.25	0.898	0.252	68.5	850MB	Yes
M	Plenoxels [12]	20.12	23.08	0.626	0.463	6.2	3.1GB	No
	Vanilla 3DGS [19]	27.05	27.21	0.815	0.214	32.5	2.1GB	No
	LightGaussian [11]	25.14	26.40	0.781	0.250	58.6	240MB	No
	DecoupleGS (Ours)	26.85	27.12	0.805	0.218	52.4	950MB	Yes
L	Plenoxels [12]	18.55	21.08	0.719	0.379	3.5	4.8GB	No
	Vanilla 3DGS [19]	25.50	25.14	0.785	0.255	12.1	3.8GB	No
	LightGaussian [11]	23.52	24.10	0.720	0.295	45.0	320MB	No
	DecoupleGS (Ours)	25.24	25.05	0.772	0.260	38.5	1.1GB	Yes

4.1 Experimental Setup

Datasets and assets. We utilize nuScenes [3] and PandaSet [39] for diverse background scenarios, and source manipulable canonical vehicles from 3DRealCar [10]. We curate 15 dynamic clips from nuScenes and 10 sequences from PandaSet, covering challenging illumination conditions such as dusk, night, and overcast scenes. Dynamic objects are removed using Segment Anything Model (SAM) [21] masks with morphological dilation to suppress boundary artifacts. We sample 20 vehicle assets from 3DRealCar, covering sedans, SUVs, and large commercial vehicles.

Baselines and metrics. For rendering efficiency and compression, we compare against Plenoxels [12], Vanilla 3DGS [19], and LightGaussian [11]. For simulator-level fidelity and closed-loop testing, we compare with HUGSIM [47], OASim [40], and RealEngine [18]. Downstream E2E policy testing is conducted with UniAD [16] and VAD [17]. Rendering fidelity is measured by PSNR-Veh, PSNR-All, SSIM, and LPIPS, while runtime performance is evaluated using FPS and VRAM. Geometric consistency is quantified by Trajectory Average Displacement Error (ADE) and Ground Penetration Rate (GPR). Photometric realism is evaluated using Masked PSNR, Peak Intensity Error (PIE), and Peak Angular Error (PAE). Open-loop planner consistency is measured by mADE and minTTC, while closed-loop E2E testing adopts Driving Score (DS), Success Rate (SR), Route Completion (RC), and minTTC.

Implementation and closed-loop protocol. All experiments are conducted on a single NVIDIA RTX 4090 GPU with 24 GB VRAM using custom CUDA kernels. The static background is optimized for 30k iterations, and the asset compression threshold τ_s is set to 0.005. All closed-loop evaluations are fully interactive. At each simulation step, DecoupleGS renders multi-view images for the tested planner, the E2E model predicts the ego trajectory, and background

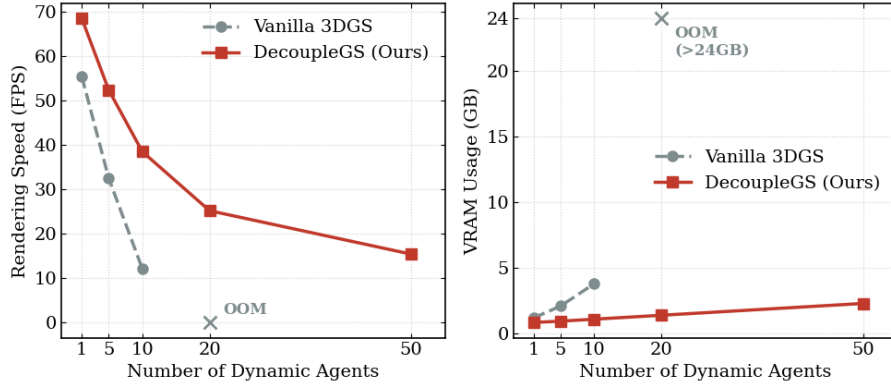


Fig. 5: Scalability evaluation. DecoupleGS avoids the OOM bottlenecks of Vanilla 3DGS, demonstrating stable VRAM growth and usable frame rates up to 50 agents.

agents react online via coupled IDM and MOBIL behavioral models. The global scene state is then synchronized and re-rendered for the next step.

4.2 Rendering Efficiency and Scalability

Asset compression. Table 1 reports results under three traffic densities: Size S with 1–2 vehicles, Size M with 3–5 vehicles, and Size L with 6–10+ vehicles. Vanilla 3DGS preserves the highest rendering fidelity but becomes inefficient as traffic density increases, while LightGaussian achieves the most aggressive compression and fastest rendering at the cost of lower scene fidelity. DecoupleGS is designed for a different objective: it preserves the high-fidelity static background and compresses only dynamic assets, yielding the most balanced fidelity-efficiency trade-off. As shown in the final column, among the compared methods, DecoupleGS is the only one that consistently remains within the top-2 across all fidelity and efficiency metrics under every traffic density, which is more suitable for interactive E2E testing than optimizing a single isolated metric.

Scalability. We further conduct a stress test by increasing the number of dynamic agents to 50. As shown in Fig. 5, Vanilla 3DGS suffers rapid memory growth and encounters out-of-memory failures before reaching dense traffic. In contrast, DecoupleGS relies on a persistent background and compact VQ-indexed assets, leading to approximately linear memory growth. It maintains usable throughput even under extreme multi-agent settings, which is essential for large-scale closed-loop policy evaluation.

4.3 Geometric and Photometric Fidelity

Map-guided geometric consistency. Fig. 6 evaluates the metric alignment of inserted agents. Compared with HUGSIM and RealEngine, our map-guided registration significantly reduces trajectory drift and ground penetration. By

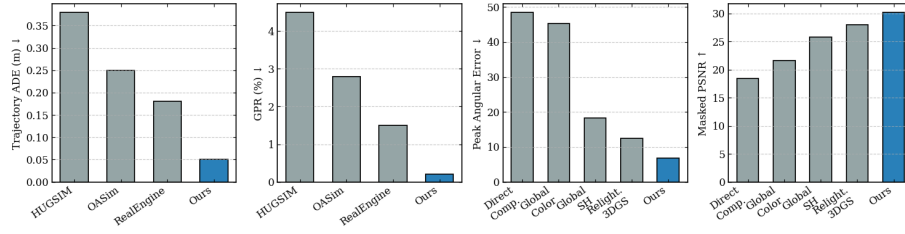


Fig. 6: Quantitative comparisons of geometric registration and photometric realism against baseline methods. DecoupleGS achieves lower trajectory drift, fewer ground-penetration artifacts, and more consistent relighting.

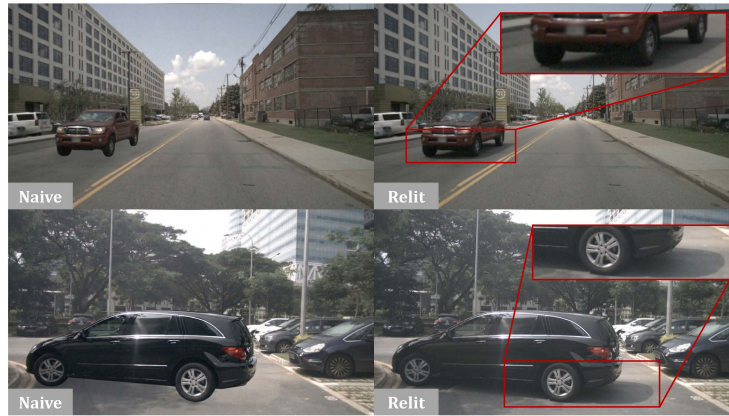


Fig. 7: Qualitative comparison of the relighting module, demonstrating seamless integration of dynamic assets under various illuminations.

combining trajectory-to-lane sequence alignment, $SE(2)$ planar correction, and opacity-accumulated vertical grounding, DecoupleGS achieves accurate lane-level placement while maintaining physically plausible contact with the road surface.

Photometric realism. The same figure also reports photometric realism under held-out vehicle re-insertion. Our proxy-based relighting obtains the lowest PAE and competitive Masked PSNR by transferring local ambient illumination to canonical assets without online neural inference. As shown in Fig. 7, inserted vehicles inherit scene-dependent lighting and contact shadows, avoiding the common mismatch between foreground appearance and background illumination.

4.4 Sim-to-real Consistency

A reliable simulator should not only render visually plausible images but also preserve the behavior of downstream planners. We therefore conduct an open-loop sim-to-real consistency test using UniAD. Real images serve as the reference input, while different simulators provide rendered observations for the same driv-

Table 2: Open-loop sim-to-real consistency and closed-loop E2E simulator comparison. Open-loop results measure UniAD behavior under rendered observations, using real images as the reference input. Bold values indicate the best result among simulated inputs. Closed-loop scores are normalized to $[0, 1]$ for consistency with Table 3.

Setting	Metric	Real Input	Ours	HUGSIM [47]	RealEngine [18]	OASim [40]
Open-loop	mADE (m) ↓	0.76	0.82	0.94	1.15	1.02
	minTTC (s) ↑	3.5	3.3	2.8	2.4	2.9
Closed-loop	Driving Score ↑	–	0.884	0.765	0.682	0.748
	Route Completion ↑	–	0.956	0.814	0.795	0.851
	minTTC (s) ↑	–	3.3	2.3	2.6	2.8
	Rendering FPS ↑	–	45	12	32	18

Table 3: Closed-loop E2E AD testing across scenario difficulty levels. Values are Mean $\pm$ Std over 50 independent episodes.

Difficulty	Method	DS	SR	RC	minTTC
Easy	UniAD [16]	0.725 $\pm$ 0.04	0.850 $\pm$ 0.03	0.780 $\pm$ 0.05	0.812 $\pm$ 0.06
	VAD [17]	0.680 $\pm$ 0.05	0.815 $\pm$ 0.04	0.720 $\pm$ 0.06	0.785 $\pm$ 0.05
Medium	UniAD [16]	0.540 $\pm$ 0.06	0.710 $\pm$ 0.05	0.610 $\pm$ 0.07	0.650 $\pm$ 0.08
	VAD [17]	0.450 $\pm$ 0.07	0.630 $\pm$ 0.06	0.520 $\pm$ 0.08	0.580 $\pm$ 0.07
Hard	UniAD [16]	0.380 $\pm$ 0.08	0.520 $\pm$ 0.07	0.450 $\pm$ 0.09	0.480 $\pm$ 0.10
	VAD [17]	0.250 $\pm$ 0.09	0.410 $\pm$ 0.08	0.320 $\pm$ 0.10	0.390 $\pm$ 0.09
Extreme	UniAD [16]	0.195 $\pm$ 0.07	0.310 $\pm$ 0.09	0.250 $\pm$ 0.08	0.280 $\pm$ 0.08
	VAD [17]	0.120 $\pm$ 0.05	0.220 $\pm$ 0.07	0.180 $\pm$ 0.06	0.210 $\pm$ 0.06

ing scenes. As shown in Table 2, DecoupleGS most closely matches the real-data behavior in both mADE and minTTC, indicating a smaller sim-to-real behavioral gap than existing neural simulators.

4.5 Closed-loop E2E Testing

Simulator-level comparison. Beyond open-loop consistency, Table 2 compares different simulators under interactive closed-loop testing. DecoupleGS achieves the highest normalized Driving Score and Route Completion while maintaining the largest safety margin measured by minTTC. It also runs at 45 FPS, substantially faster than HUGSIM and OASim. These results demonstrate that DecoupleGS is not merely a rendering module, but a practical closed-loop sensor simulation engine for E2E policy evaluation.

Difficulty-based stress testing. We further evaluate UniAD and VAD across four scenario difficulty levels, as shown in Table 3, with representative closed-loop visualizations provided in Fig. 8. The difficulty is controlled by traffic density, illumination, and background-agent aggressiveness. Easy scenarios contain sparse traffic and clear daytime illumination. Medium scenarios introduce moderate density and standard urban maneuvers. Hard scenarios include dense



Fig. 8: Representative camera views of UniAD evaluated across generated scenarios of increasing difficulty, from Easy to Extreme.

Table 4: System-level ablation study evaluated on medium-density scenarios. Values in parentheses indicate changes relative to the full framework.

Methods	FPS $\uparrow$	Trajectory ADE $\downarrow$	Peak Angular Error $\downarrow$
w/o asset compression	8.5 ($\downarrow 26.9$)	0.05 (=)	6.8 (=)
w/o map-guided reg.	36.2 ($\uparrow 0.8$)	0.48 ($\uparrow 0.43$)	6.8 (=)
w/o relighting	38.5 ($\uparrow 3.1$)	0.05 (=)	48.5 ($\uparrow 41.7$)
Full Framework (Ours)	35.4	0.05	6.8

traffic and challenging illumination such as dusk or heavy overcast. Extreme scenarios further introduce adversarial lighting and safety-critical behaviors such as sudden braking and aggressive cut-ins.

Both planners degrade as the scenario difficulty increases. UniAD maintains a relatively high completion rate in Easy scenarios but suffers under severe occlusion and dense interactions, while VAD is more sensitive to dense multi-agent constraints, causing a steeper drop in Driving Score under Hard and Extreme settings. The qualitative examples in Fig. 8 further illustrate that DecoupleGS can synthesize visually diverse and increasingly challenging closed-loop scenarios, including dense traffic, low-light conditions, and safety-critical interactions. These results show that DecoupleGS can generate structured and controllable stress tests that expose planner vulnerabilities.

4.6 Ablation Studies

Table 4 decomposes the effect of each core module under medium-density scenarios. Without asset compression, throughput collapses from 35.4 FPS to 8.5 FPS, making interactive multi-agent simulation impractical. Without map-guided registration, ADE increases from 0.05 m to 0.48 m, showing that neural coordinates alone do not guarantee metric-consistent placement. Without relighting, PAE in-

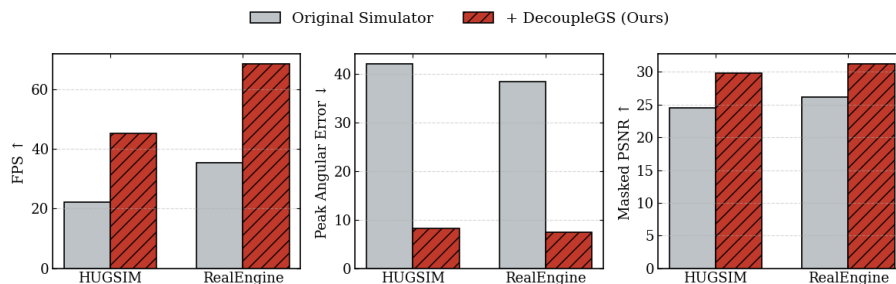


Fig. 9: Plug-and-play capability. Integrating our modules into existing simulators improves rendering speed and photometric accuracy.

creases from 6.8° to 48.5° , exposing severe foreground-background illumination mismatch. The full framework therefore offers the best balanced trade-off across efficiency, geometric alignment, and photometric consistency.

Plug-and-play capability. We further validate that the proposed modules are solver-agnostic. As shown in Fig. 9, integrating DecoupleGS components into existing simulators improves runtime and photometric consistency, indicating that our compression, registration, and relighting modules can serve as general-purpose building blocks for interactive neural driving simulation.

4.7 Limitations

While DecoupleGS enables efficient and consistent closed-loop testing, it still has limitations. First, its proxy-based relighting favors runtime efficiency over physically exact global illumination, and may struggle with complex multi-bounce lighting, specular effects, and cast shadows. Second, map-guided registration relies on reliable semantic topology, making multi-level roads, overpasses, under-bridge regions, and poorly detected lanes more challenging. Third, the current framework mainly targets rigid traffic agents and camera simulation. Future work will explore non-rigid foregrounds, full background compression, richer interactive behaviors, and unified camera-LiDAR sensor simulation.

5 Conclusion

This paper presents DecoupleGS, a decoupled 3DGS framework for interactive E2E autonomous driving testing. By separating persistent backgrounds from manipulable agents, DecoupleGS addresses efficiency, geometric, and photometric conflicts through asset compression, map-guided registration, and proxy-based relighting. Experiments demonstrate a balanced fidelity-efficiency trade-off, improved sim-to-real consistency, and fully interactive closed-loop policy evaluation. Future work will extend DecoupleGS toward richer illumination, non-rigid agents, and unified RGB-LiDAR simulation.

Acknowledgements

This work was supported by the National Natural Science Foundation of China (Grant No. 52502514) and the Fundamental Research Funds for the Central Universities.

References

1. Barron, J.T., Mildenhall, B., Verbin, D., Srinivasan, P.P., Hedman, P.: Mip-NeRF 360: Unbounded anti-aliased neural radiance fields. In: *IEEE Conf. Comput. Vis. Pattern Recog.* (2022)
2. Behrisch, M., Bieker, L., Erdmann, J., Krajzewicz, D.: SUMO: Simulation of urban mobility. In: *Int. Conf. Advances Syst. Simul.* (2011)
3. Caesar, H., Bankiti, V., Lang, A.H., Vora, S., Liong, V.E., Xu, Q., Krishnan, A., Pan, Y., Baldan, G., Beijbom, O.: nuScenes: A multimodal dataset for autonomous driving. In: *IEEE Conf. Comput. Vis. Pattern Recog.* (2020)
4. Cao, W., Hallgarten, M., Li, T., Dauner, D., Gu, X., Wang, C., Miron, Y., Aiello, M., Li, H., Gilitschenski, I., Ivanovic, B., Pavone, M., Geiger, A., Chitta, K.: Pseudo-simulation for autonomous driving. In: *Conf. Robot Learn. Proceedings of Machine Learning Research*, vol. 305, pp. 4709–4722. PMLR (2025)
5. Chen, A., Xu, Z., Geiger, A., Yu, J., Su, H.: TensorRF: Tensorial radiance fields. In: *Eur. Conf. Comput. Vis.* (2022)
6. Chen, L., Wu, P., Chitta, K., Jaeger, B., Geiger, A., Li, H.: End-to-end autonomous driving: Challenges and frontiers. *IEEE Trans. Pattern Anal. Mach. Intell.* **46**(12), 10164–10183 (2024)
7. Chitta, K., Prakash, A., Jaeger, B., Yu, Z., Renz, K., Geiger, A.: Transfuser: Imitation with transformer-based sensor fusion for autonomous driving. *IEEE Trans. Pattern Anal. Mach. Intell.* **45**, 12878–12895 (2023)
8. Codevilla, F., Müller, M., López, A., Koltun, V., Dosovitskiy, A.: End-to-end driving via conditional imitation learning. In: *IEEE Int. Conf. Robot. Autom.* (2018)
9. Dosovitskiy, A., Ros, G., Codevilla, F., Lopez, A., Koltun, V.: CARLA: An open urban driving simulator. In: *Conf. Robot Learn.* (2017)
10. Du, X., Wang, Y., Sun, H., Wu, Z., Sheng, H., Wang, S., Ying, J., Lu, M., Zhu, T., Zhan, K., Yu, X.: 3DRealCar: An in-the-wild RGB-D car dataset with 360-degree views. In: *Int. Conf. Comput. Vis.* pp. 26488–26498 (2025)
11. Fan, Z., Wang, K., Wen, K., Zhu, Z., Xu, D., Wang, Z.: LightGaussian: Unbounded 3D gaussian compression with 15x reduction and 200+ FPS. In: *Adv. Neural Inform. Process. Syst.* (2024). <https://doi.org/10.52202/079017-4447>
12. Fridovich-Keil, S., Yu, A., Tancik, M., Chen, Q., Recht, B., Kanazawa, A.: Plenoxels: Radiance fields without neural networks. In: *IEEE Conf. Comput. Vis. Pattern Recog.* (2022)
13. Fridovich-Keil, S., et al.: K-planes: Explicit radiance fields in space, time, and appearance. In: *IEEE Conf. Comput. Vis. Pattern Recog.* (2023)
14. Ge, J., Liu, Z., Fan, L., Jiang, Y., Su, J., Li, Y., Zhang, Z., Chen, S.: Unraveling the effects of synthetic data on end-to-end autonomous driving. In: *Int. Conf. Comput. Vis.* pp. 28859–28869 (2025)
15. Hu, A., Russell, L., Yeo, H., et al.: GAIA-1: A generative world model for autonomous driving. *arXiv preprint arXiv:2309.17080* (2023)

16. Hu, Y., Yang, J., Chen, L., Li, K., Sima, C., Zhu, X., Chai, S., Du, S., Lin, T., Wang, W., et al.: Planning-oriented autonomous driving. In: IEEE Conf. Comput. Vis. Pattern Recog. (2023)
17. Jiang, B., Chen, S., Xu, Q., Liao, B., Chen, J., Zhou, H., Zhang, Q., Liu, W., Huang, C., Wang, X.: VAD: Vectorized scene representation for efficient autonomous driving. In: Int. Conf. Comput. Vis. (2023)
18. Jiang, J., Song, N., Li, J., Zhu, X., Zhang, L.: RealEngine: Simulating autonomous driving in realistic context. arXiv preprint arXiv:2505.16902 (2025)
19. Kerbl, B., Kopanas, G., Leimkühler, T., Drettakis, G.: 3D gaussian splatting for real-time radiance field rendering. ACM Trans. Graph. **42**(4) (2023)
20. Kesting, A., Treiber, M., Helbing, D.: General lane-changing model MOBIL for car-following models. Transportation Research Record **1999**(1), 86–94 (2007)
21. Kirillov, A., Mintun, E., Ravi, N., Mao, H., Rolland, C., Gustafson, L., Xiao, T., Whitehead, S., Berg, A.C., Lo, W.Y., Dollar, P., Girshick, R.: Segment anything. In: Int. Conf. Comput. Vis. pp. 4015–4026 (2023)
22. Kundu, A., et al.: Panoptic neural fields: A semantic object-aware neural scene representation. In: IEEE Conf. Comput. Vis. Pattern Recog. (2022)
23. Li, Q., Peng, Z., Feng, L., Zhang, Q., Xue, Z., Zhou, B.: Metadrive: Composing diverse driving scenarios for generalizable reinforcement learning. IEEE Trans. Pattern Anal. Mach. Intell. **45**(3), 3461–3475 (2023)
24. Liao, B., Chen, S., Zhang, Y., et al.: MapTRv2: An end-to-end framework for online vectorized HD map construction. Int. J. Comput. Vis. **133**(3), 1352–1374 (2025)
25. Liu, Z., Fu, D., Cai, P., Wang, L., Liu, Y., Ren, Y., Shi, B., Wang, J.: SymDrive: Realistic and controllable driving simulator via symmetric auto-regressive online restoration. arXiv preprint arXiv:2512.21618 (2025)
26. Ljungbergh, W., Tonderski, A., Johnander, J., Caesar, H., Åström, K., Felsberg, M., Petersson, C.: NeuroNCAP: Photorealistic closed-loop safety testing for autonomous driving. In: Eur. Conf. Comput. Vis. pp. 161–177 (2024)
27. Mildenhall, B., Srinivasan, P.P., Tancik, M., Barron, J.T., Ramamoorthi, R., Ng, R.: NeRF: Representing scenes as neural radiance fields for view synthesis. In: Eur. Conf. Comput. Vis. (2020)
28. Müller, T., Evans, A., Schied, C., Keller, A.: Instant neural graphics primitives with a multiresolution hash encoding. ACM Trans. Graph. **41**(4) (2022)
29. Ost, J., et al.: Neural scene graphs for dynamic scenes. In: IEEE Conf. Comput. Vis. Pattern Recog. (2021)
30. Rombach, R., Blattmann, A., Lorenz, D., Esser, P., Ommer, B.: High-resolution image synthesis with latent diffusion models. In: IEEE Conf. Comput. Vis. Pattern Recog. (2022)
31. Rong, G., Shin, B.H., Tabatabaee, H., Lu, Q., Lemke, S., Možeiko, M., Boise, E., Uhm, G., Gerow, M., Mehta, S., Agafonov, E., Kim, T.H., Sterner, E., Ushiroda, K., Reyes, M., Zelenkovsky, D., Kim, S.: LGSVL simulator: A high fidelity simulator for autonomous driving. In: IEEE Intell. Transp. Syst. Conf. pp. 1–6 (2020). <https://doi.org/10.1109/ITSC45102.2020.9294422>
32. Shah, S., Dey, D., Lovett, C., Kapoor, A.: AirSim: High-fidelity visual and physical simulation for autonomous vehicles. In: Field and Service Robotics (2018)
33. Tampuu, A., Matiisen, T., Semikin, M., Fishman, D., Muhammad, N.: A survey of end-to-end driving: Architectures and training methods. IEEE Trans. Neural Netw. Learn. Syst. **33**(4), 1364–1384 (2022)
34. Tancik, M., et al.: Block-nerf: Scalable large scene neural view synthesis. In: IEEE Conf. Comput. Vis. Pattern Recog. (2022)

35. Treiber, M., Hennecke, A., Helbing, D.: Congested traffic states in empirical observations and microscopic simulations. *Physical Review E* **62**(2), 1805–1824 (2000)
36. Turki, H., et al.: Mega-nerf: Scalable construction of large-scale nerfs for virtual fly-throughs. In: *IEEE Conf. Comput. Vis. Pattern Recog.* (2022)
37. Wang, X., Zhu, Z., Huang, G., Chen, X., Zhu, J., Lu, J.: DriveDreamer: Towards real-world-drive world models for autonomous driving. In: *Eur. Conf. Comput. Vis.* pp. 55–72 (2024). https://doi.org/10.1007/978-3-031-73195-2_4
38. Wu, Z., Liu, T., Luo, L., Zhong, Z., Chen, J., Xiao, H., Hou, C., Lou, H., Chen, Y., Yang, R., et al.: MARS: An instance-aware, modular and realistic simulator for autonomous driving. In: *Artificial Intelligence. Lecture Notes in Computer Science*, vol. 14473, pp. 3–15 (2023). https://doi.org/10.1007/978-981-99-8850-1_1
39. Xiao, P., Shao, Z., Hao, S., Zhang, Z., Chai, X., Jiao, J., Li, Z., Wu, J., Sun, K., Jiang, K., Wang, Y., Yang, D.: PandaSet: Advanced sensor suite dataset for autonomous driving. In: *IEEE Intell. Transp. Syst. Conf.* (2021)
40. Yan, G., Pi, J., Guo, J., Luo, Z., Dou, M., Deng, N., Huang, Q., Fu, D., Wen, L., Cai, P., Gao, X., Cai, X., Zhang, B., Yang, X., Bai, Y., Zhou, H., Shi, B.: OASim: An open and adaptive simulator based on neural rendering for autonomous driving. *arXiv preprint arXiv:2402.03830* (2024)
41. Yan, T., Wu, D., Han, W., Jiang, J., Zhou, X., Zhan, K., Xu, C.Z., Shen, J.: DrivingSphere: Building a high-fidelity 4D world for closed-loop simulation. In: *IEEE Conf. Comput. Vis. Pattern Recog.* pp. 27531–27541 (2025)
42. Yan, Y., Lin, H., Zhou, C., Wang, W., Sun, H., Zhan, K., Lang, X., Zhou, X., Peng, S.: Street Gaussians: Modeling dynamic urban scenes with gaussian splatting. In: *Eur. Conf. Comput. Vis.* (2024)
43. Yan, Z., Low, W.F., Chen, Y., Lee, G.H.: Multi-scale 3D gaussian splatting for anti-aliased rendering. In: *IEEE Conf. Comput. Vis. Pattern Recog.* pp. 20923–20931 (2024)
44. Yang, X., Wen, L., Wei, T., Ma, Y., Mei, J., Li, X., Lei, W., Fu, D., Cai, P., Dou, M., He, L., Liu, Y., Shi, B., Qiao, Y.: DriveArena: A closed-loop generative simulation platform for autonomous driving. In: *Int. Conf. Comput. Vis.* pp. 26933–26943 (2025)
45. Yang, Z., Chen, Y., Wang, J., Manivasagam, S., Ma, W.C., Yang, A.J., Urtasun, R.: UniSim: A neural closed-loop sensor simulator. In: *IEEE Conf. Comput. Vis. Pattern Recog.* pp. 1389–1399 (2023)
46. Yu, H.X., et al.: Unsupervised discovery of object radiance fields. In: *Int. Conf. Learn. Represent.* (2022)
47. Zhou, H., Lin, L., Wang, J., Lu, Y., Bai, D., Liu, B., Wang, Y., Geiger, A., Liao, Y.: HUGSIM: A real-time, photo-realistic and closed-loop simulator for autonomous driving. *IEEE Trans. Pattern Anal. Mach. Intell.* **48**(4), 4673–4691 (2026). <https://doi.org/10.1109/TPAMI.2025.3647952>
48. Zhou, X., Lin, Z., Shan, X., Wang, Y., Sun, D., Yang, M.H.: DrivingGaussian: Composite gaussian splatting for surrounding dynamic autonomous driving scenes. In: *IEEE Conf. Comput. Vis. Pattern Recog.* pp. 21634–21643 (2024)
49. Zhu, X., Yi, R., Wen, X., Zhu, C., Xu, K.: Relighting scenes with object insertions in neural radiance fields. *IEEE Trans. Circuit Syst. Video Technol.* **35**(7), 6787–6802 (2025)

DecoupleGS: Interactive 3D Gaussian Splatting for End-to-End Autonomous Driving Testing

Supplementary Material

Overview

This supplementary document provides additional details, theoretical expansions, and extended experimental results to support the main paper *DecoupleGS: Interactive 3D Gaussian Splatting for End-to-End Autonomous Driving Testing*. The material is structured as follows:

- **Section A:** Detailed implementation configurations, including system overhead, training schedules, specific hyperparameters for both background reconstruction and asset compression, and dataset curation strategies.
- **Section B:** In-depth mathematical formulations for the Vector Quantization (VQ) optimization, proxy-based relighting calibration, and contact shadow generation, alongside comprehensive algorithmic pseudocodes for the offline and online pipelines.
- **Section C:** Concrete definitions of the evaluation scenario difficulties (Easy to Extreme), rigorous mathematical formulations of the evaluation metrics, and complete details regarding the interactive closed-loop system integration, including the coupled Intelligent Driver Model (IDM) [35] and Minimizing Overall Braking Induced by Lane change (MOBIL) [20] behavioral models for background agents.
- **Section D:** Extended qualitative results demonstrating the framework’s robust capabilities in multi-view scene decoupling and interactive scene editing.

A Implementation Details

In this section, we provide the comprehensive engineering configurations required to reproduce the DecoupleGS framework, elaborating on the concise descriptions presented in Section 4.2 of the main text.

A.1 Training Schedules and System Overhead

Our pipeline consists of two primary offline optimization stages: background scene reconstruction and canonical asset compression. The framework is implemented in PyTorch, utilizing custom CUDA kernels for efficient unified rasterization and bounding-box level frustum culling. All optimizations and evaluations are conducted on a single NVIDIA RTX 4090 GPU (24 GB VRAM) alongside an Intel Core i9-13900K CPU.

Table 5: Hyperparameters for Static Background 3DGS Optimization.

Parameter	Value
Total Iterations	30,000
Position Learning Rate (μ)	1.6×10^{-4} decaying to 1.6×10^{-6}
Feature Learning Rate (c)	2.5×10^{-3}
Opacity Learning Rate (α)	0.05
Scaling Learning Rate (s)	5.0×10^{-3}
Densification Interval	Every 100 iterations (from iter 500 to 15,000)
Densification Gradient Threshold	2.0×10^{-4}
Opacity Reset Interval	Every 3,000 iterations

Static Background Reconstruction. The background environment is optimized using the vanilla 3D Gaussian Splatting (3DGS) [19] pipeline for 30,000 iterations. We employ the Adam optimizer with a dynamic learning rate. Table 5 summarizes the detailed hyperparameter settings. On average, optimizing a single background scenario (approx. 200 frames) takes roughly 45 minutes.

Semantic-aware Asset Compression. For dynamic traffic agents, canonical Gaussians are first optimized for 20,000 iterations. Unlike the unbounded background, vehicle primitives are strictly constrained within a scaled 3D bounding box to prevent floaters. The densification gradient threshold is slightly relaxed to 4.0×10^{-4} to avoid excessive primitive generation inside the vehicle cabin. Offline training per vehicle asset takes approximately 15 minutes.

Subsequently, explicit importance scoring and adaptive pruning are applied. The pruning threshold τ_s is strictly set to 0.005. The weighting coefficients in Equation (4) of the main text were determined via grid search over the discrete space $\{0.1, 0.3, 0.5, 0.7, 0.9\}$. The optimal configuration was found to be: $w_{\text{vis}} = 0.5$, $w_{\text{col}} = 0.3$, and $w_{\text{ent}} = 0.2$. This configuration strongly prioritizes the preservation of visible structural contours while moderately suppressing internal unobservable primitives.

For attribute VQ, we maintain codebooks of size $K_{\text{color}} = 1024$ for Spherical Harmonics (SH) features and $K_{\text{shape}} = 512$ for covariances. Crucially, to ensure convergence, the codebooks are initialized using K-Means clustering over the pruned primitives rather than random uniform sampling. They are then dynamically updated during the final 5,000 iterations using Exponential Moving Average (EMA) with a momentum factor of $\gamma = 0.99$.

A.2 Map-guided Registration Parameters

For the geometric registration module (Section 3.3 in the main text), the Dynamic Time Warping (DTW) cost matrix $C_{t,u}$ is constructed by balancing Euclidean distance and heading mismatch. The scalar weight for the heading penalty is empirically set to $\lambda_{\text{heading}} = 2.5$, ensuring that vehicles strictly adhere to lane orientations during sharp turns or lane changes. For scenarios where HD-Map are not provided, MapTRv2 [24] is deployed with its default pre-

trained weights on nuScenes [3], and the extracted polylines are rasterized at a 0.1m/pixel resolution for the Orthogonal Procrustes analysis.

A.3 Dataset Curations

To rigorously evaluate DecoupleGS across diverse and challenging conditions, we carefully curated subsets from multiple large-scale autonomous driving datasets.

Background Scenarios. We selected 15 dynamic clips (each roughly 20 seconds at 10 Hz) from the **nuScenes** dataset, explicitly choosing high-complexity sequences. Additionally, we extracted 10 sequences from **PandaSet** [39] to cover adversarial illumination conditions (dusk, night, and overcast). To obtain clean background plates, dynamic objects within these sequences were masked out using the Segment Anything Model (SAM) [21] with a morphological dilation of 5 pixels to prevent edge artifacts.

Manipulable Canonical Assets. The canonical vehicle instances were sourced from the **3DRealCar** [10] dataset. We systematically sampled 20 distinct vehicle models to ensure a diverse morphological distribution: Sedans (10 instances), SUVs (6 instances), and Large Commercial Vehicles (4 instances).

B Methodological Expansions

In this section, we provide the complete mathematical formulations for the VQ optimization and the proxy-based relighting operators, which were briefly introduced in Section 3 of the main manuscript.

B.1 Vector Quantization Optimization via EMA

As described in Section 3.2 of the main text, to overcome the memory bottleneck of storing multiple vehicle instances, we apply VQ to the high-dimensional attributes (covariance Σ_i and SH color coefficients c_i). Instead of a static quantization, our codebooks $\mathcal{C}_{\text{shape}}$ and $\mathcal{C}_{\text{color}}$ are dynamically optimized using an EMA to prevent index collapse and maintain representation fidelity.

Let $e_k^{(t)}$ denote the k -th entry in the codebook at iteration t , and x_i denote the input continuous attribute of the i -th Gaussian. In the forward pass, each primitive is assigned to its nearest codebook entry:

$$q_i = \arg \min_k \|x_i - e_k^{(t)}\|_2^2 \quad (10)$$

To update the codebook without requiring differentiable backpropagation through the argmin operation, we track the cluster size N_k and the sum of assigned vectors m_k . At each optimization step, we update these statistics using EMA with a momentum factor $\gamma \in [0, 1)$ (we set $\gamma = 0.99$):

$$N_k^{(t)} = \gamma N_k^{(t-1)} + (1 - \gamma) \sum_i \mathbb{I}(q_i = k) \quad (11)$$

$$m_k^{(t)} = \gamma m_k^{(t-1)} + (1 - \gamma) \sum_{i: q_i = k} x_i \quad (12)$$

where $\mathbb{I}(\cdot)$ is the indicator function. The codebook entry is then explicitly updated as the smoothed centroid:

$$e_k^{(t)} = \frac{m_k^{(t)}}{N_k^{(t)}} \quad (13)$$

To handle dead codes (entries where $N_k^{(t)} \approx 0$), we periodically re-initialize them using K-Means++ over a random subset of the current active primitives x_i , ensuring maximum utilization of the codebook capacity.

B.2 Proxy-based Relighting Analytical Calibration

A core contribution of DecoupleGS is the ability to transfer ambient illumination to canonical assets without online MLP inference. In Equation (8) of the main text, we defined the linear modulation:

$$c_{\text{out}} = \mathcal{T}(\mathcal{L}(x)) \otimes c_{\text{can}} + b(\mathcal{L}(x)) \quad (14)$$

Here, we clarify the derivation of the operators $\mathcal{T}(\cdot)$ and $b(\cdot)$. To maintain real-time performance, we parameterize these operators as affine functions of the local ambient descriptor $\mathcal{L}(x) \in \mathbb{R}^{27}$ (aggregating the first 3 bands of SH coefficients from the background):

$$\mathcal{T}(\mathcal{L}) = \mathbf{W}_T \mathcal{L} + \mathbf{w}_0, \quad b(\mathcal{L}) = \mathbf{W}_B \mathcal{L} + \mathbf{b}_0 \quad (15)$$

To avoid online optimization, we perform an offline analytical calibration. We generate a synthetic calibration dataset by rendering the canonical asset under N diverse, pre-captured HDRI lighting probes. For each primitive i , we extract its canonical color $c_{\text{can},i}$ and its ground-truth relit color $c_{\text{out},i}^{(n)}$ under the n -th probe descriptor $\mathcal{L}^{(n)}$.

We then formulate this as a global Ordinary Least Squares (OLS) regression problem over the dataset $\{c_{\text{can},i}, \mathcal{L}^{(n)}, c_{\text{out},i}^{(n)}\}$. The objective is to find the optimal global parameter matrices $\Theta = \{\mathbf{W}_T, \mathbf{w}_0, \mathbf{W}_B, \mathbf{b}_0\}$ that minimize the photometric residual:

$$\min_{\Theta} \sum_{n=1}^N \sum_{i \in \mathcal{V}} \left\| \left(\mathbf{W}_T \mathcal{L}^{(n)} + \mathbf{w}_0 \right) \otimes c_{\text{can},i} + \left(\mathbf{W}_B \mathcal{L}^{(n)} + \mathbf{b}_0 \right) - c_{\text{out},i}^{(n)} \right\|_2^2 \quad (16)$$

Since this objective is quadratic with respect to the parameters in Θ , it admits a closed-form analytical solution via the normal equations. Once computed offline, Θ is stored globally. During online E2E testing, relighting a vehicle only requires two efficient matrix-vector multiplications per vehicle frame, completely bypassing the computational bottleneck of evaluating neural networks on millions of primitives.

B.3 Parametric Contact Shadow Generation

To ground the vehicle visually, Equation (9) introduces the shadow modulation function $1 - \lambda \cdot I_{\text{dom}} \cdot M_{\text{shadow}}(u)$. We define the spatial shadow mask $M_{\text{shadow}}(u)$ as a 2D super-ellipse projected onto the estimated ground plane, smoothly decaying outward. Let $u = (u_x, u_y)$ be the local ground-plane coordinates relative to the vehicle center, and (L, W) be the length and width of the vehicle’s bounding box footprint. The mask is defined as:

$$M_{\text{shadow}}(u) = \exp \left(- \left(\left| \frac{u_x}{L/2} \right|^p + \left| \frac{u_y}{W/2} \right|^p \right)^{\frac{1}{p}} \cdot \tau_{\text{decay}} \right) \quad (17)$$

where $p = 4$ creates a rounded rectangular shape matching the vehicle chassis, and τ_{decay} controls the softness of the shadow edge. The dominant light intensity $I_{\text{dom}} = \max(0, \mathcal{L}_{\text{DC}}(x))$ dynamically scales this mask, ensuring shadows are stark under direct sunlight (high I_{dom}) and faint in overcast or night scenarios (low I_{dom}).

Algorithm 1 Offline Semantic-aware Asset Compression

Require: Raw canonical 3DGS vehicle $\mathcal{V}_{\text{raw}}$, pruning threshold τ_s , EMA momentum γ

Ensure: Compact canonical volume $\mathcal{V}_{\text{compact}}$ and Codebooks $\mathcal{C}_{\text{shape}}, \mathcal{C}_{\text{color}}$

- 1: Initialize codebooks $\mathcal{C}_{\text{shape}}, \mathcal{C}_{\text{color}}$ via K-Means++ on raw attributes
 - 2: **for** each primitive $g_i \in \mathcal{V}_{\text{raw}}$ **do**
 - 3: Calculate visibility V_i , color contrast D_i , and texture entropy H_i
 - 4: Compute scalar importance $s_i = w_{\text{vis}}V_i + w_{\text{col}}D_i + w_{\text{ent}}H_i$
 - 5: **if** $s_i < \tau_s$ **then**
 - 6: Prune g_i from $\mathcal{V}_{\text{raw}}$
 - 7: **end if**
 - 8: **end for**
 - 9: **for** EMA iteration $t = 1$ to 5000 **do**
 - 10: **for** each retained primitive g_i **do**
 - 11: Find nearest indices $q_i^{\text{shape}}, q_i^{\text{color}}$ in $\mathcal{C}_{\text{shape}}, \mathcal{C}_{\text{color}}$
 - 12: Update cluster assignments and sizes
 - 13: **end for**
 - 14: Update $\mathcal{C}_{\text{shape}}, \mathcal{C}_{\text{color}}$ using EMA with momentum γ
 - 15: **end for**
 - 16: Store $\mathcal{V}_{\text{compact}}$ using quantized indices $q_i^{\text{shape}}, q_i^{\text{color}}$
-

B.4 Algorithmic Overview of the DecoupleGS Pipeline

To provide a clear, step-by-step understanding of our decoupled framework, we summarize the core operations in two algorithms. Algorithm 1 details the offline semantic-aware asset compression. Algorithm 2 outlines the online closed-loop scenario generation, demonstrating how map-guided registration and proxy-based relighting seamlessly integrate dynamic agents into the unified rasterizer without neural inference.

Algorithm 2 Online Decoupled Integration & Unified Rendering

Require: Static background $\mathcal{S}_{\text{bg}}$, HD Map Lane $\mathcal{L}$, Asset Library $\{\mathcal{V}_k\}$, Planned Trajectory $\mathcal{T} = \{p_t\}$

Ensure: Synthesized image I_t for E2E policy evaluation

- 1: **Map-guided Registration:**
- 2: Compute alignment cost matrix $C_{t,u}$ between $\mathcal{T}$ and $\mathcal{L}$
- 3: Extract optimal warping path via DTW and compute 2D rigid transform $T_{\text{align}} \in SE(2)$
- 4: Calculate opacity-weighted vertical height z_{asset} via ray accumulation
- 5: Output fully registered 6-DoF pose $\mathcal{T}_k(t) = [\mathbf{R}_k | \mathbf{t}_k]$
- 6: **Proxy-based Relighting:**
- 7: Sample local ambient descriptor $\mathcal{L}(x)$ from neighboring $\mathcal{S}_{\text{bg}}$ primitives
- 8: Modulate asset SH coefficients: $c_{\text{out}} \leftarrow \mathcal{T}(\mathcal{L}(x)) \otimes c_{\text{can}} + b(\mathcal{L}(x))$
- 9: Generate dynamic shadow mask M_{shadow} scaled by I_{dom}
- 10: **Unified Rasterization:**
- 11: Apply spatial transforms: $\mu^{(W)} \leftarrow \mathbf{R}_k \mu^{(L)} + \mathbf{t}_k$, $\Sigma^{(W)} \leftarrow \mathbf{R}_k \Sigma^{(L)} \mathbf{R}_k^\top$
- 12: Aggregate all primitives: $\mathcal{P} \leftarrow \mathcal{S}_{\text{bg}} \cup \mathcal{V}_k$
- 13: Perform depth-sorting and continuous α -blending to output frame I_t

C Extended Experimental Setups

In Section 4.5 of the main text, we presented the closed-loop evaluation of two state-of-the-art End-to-End (E2E) autonomous driving planners, UniAD [16] and VAD [17]. To fully demonstrate the robustness and interactivity of our DecoupleGS framework, we detail the scenario definitions, the mathematical formulation of our metrics, the background agent behavioral engine, and the complete interactive closed-loop evaluation pipeline in this section.

C.1 Definition of Evaluation Scenarios

To systematically stress-test the E2E models, we categorized the generated simulation scenarios into four distinct difficulty levels based on traffic density, environmental illumination, and behavioral complexity of the background agents. Each episode runs for up to 20 seconds at a control frequency of 2Hz (matching the standard nuScenes planner configuration).

- **Easy:** Clear daytime illumination with favorable visibility. Sparse traffic density (1 to 3 dynamic agents within a 50m radius). The background agents exhibit stable, predictable behaviors (e.g., constant speed lane keeping).
- **Medium:** Daytime or overcast illumination causing mild shadow artifacts. Moderate traffic density (4 to 8 dynamic agents). Scenarios include standard urban maneuvers such as yielding at intersections and smooth lane changes by background traffic.
- **Hard:** Challenging illumination conditions, including dusk, dawn, or heavy overcast skies, demanding robust perception. Dense traffic (9 to 15 dynamic agents). Background agents perform complex maneuvers, including congested intersection crossings and close-proximity cut-ins.

- **Extreme:** Adversarial lighting conditions such as night-time rendering with extreme low-light constraints or blinding specular glare. Extremely dense traffic (> 15 dynamic agents). The background behavioral engine deliberately injects safety-critical behaviors, such as sudden hard braking by the lead vehicle or aggressive swerving, forcing the ego-vehicle into high-risk interactions.

C.2 Detailed Evaluation Metrics Formulation

As absolute 3D ground truth is intractable for in-the-wild datasets, we present the exact mathematical formulations of our targeted metrics to ensure rigorous evaluation.

Photometric Realism Metrics. We employ a hold-out strategy using precise SAM masks M . Let $I(u)$ be the synthesized color at pixel u , and $\hat{I}(u)$ be the ground-truth color. **Masked Peak Signal-to-Noise Ratio (PSNR)** evaluates the fidelity strictly within the dynamic asset’s footprint:

$$\text{Masked PSNR} = 10 \cdot \log_{10} \left(\frac{\text{MAX}_I^2}{\frac{1}{|M|} \sum_{u \in M} \|I(u) - \hat{I}(u)\|_2^2} \right) \quad (18)$$

Peak Intensity Error (PIE) measures the maximum absolute error in luminance (grayscale intensity) between the synthesized image and the ground-truth image. This captures the worst-case localized illumination magnitude mismatch. Let $Y(\cdot)$ denote the grayscale luminance channel extraction:

$$\text{PIE} = \max_{u \in M} \left| Y(I(u)) - Y(\hat{I}(u)) \right| \quad (19)$$

Peak Angular Error (PAE) explicitly isolates environmental illumination color shifts from pure intensity changes:

$$\text{PAE} = \max_{u \in M} \arccos \left(\frac{I(u) \cdot \hat{I}(u)}{\|I(u)\|_2 \|\hat{I}(u)\|_2} \right) \quad (20)$$

Geometric Alignment Metrics. Trajectory Average Displacement Error (ADE) measures the lateral drift. Given the 2D projected trajectory $\mathcal{T} = \{p_t\}_{t=1}^T$ and the semantic lane centerlines $\mathcal{L}$:

$$\text{ADE} = \frac{1}{T} \sum_{t=1}^T \min_{l \in \mathcal{L}} \|p_t - l\|_2 \quad (21)$$

Ground Penetration Rate (GPR) evaluates terrain alignment. For a vehicle with N_a bottom anchor points, let $z_{j,t}$ be the predicted vertical height, and $z_{\text{ground}}(x, y)$ be the local opacity-accumulated ground plane:

$$\text{GPR} = \frac{1}{T \cdot N_a} \sum_{t=1}^T \sum_{j=1}^{N_a} \mathbb{I}(z_{j,t} < z_{\text{ground}}(x_{j,t}, y_{j,t})) \quad (22)$$

Algorithm 3 IDM Acceleration Update

Require: Ego state (v, x, L_{car}) , Leader state $(v_{\text{lead}}, x_{\text{lead}})$, Parameters $(v_0, T, a_{\text{max}}, b_{\text{comf}}, \delta, s_0)$

Ensure: Longitudinal acceleration a_{out}

- 1: **if** Leader exists in the current lane **then**
- 2: $s \leftarrow \max(x_{\text{lead}} - x - L_{\text{car}}, 0.1)$ {Prevent zero-division in crash edge-cases}
- 3: $\Delta v \leftarrow v - v_{\text{lead}}$
- 4: **else**
- 5: $s \leftarrow 10000.0$, $\Delta v \leftarrow 0.0$ {Free road assumption}
- 6: **end if**
- 7: $s^* \leftarrow s_0 + vT + \frac{v\Delta v}{2\sqrt{a_{\text{max}}b_{\text{comf}}}}$
- 8: $a_{\text{out}} \leftarrow a_{\text{max}} \left(1 - \left(\frac{v}{v_0} \right)^\delta - \left(\frac{s^*}{s} \right)^2 \right)$
- 9: **return** a_{out}

E2E Policy Evaluation Metrics. Route Completion (RC) is the distance navigated D_{traveled} divided by the total planned distance D_{planned} : $\text{RC} = D_{\text{traveled}}/D_{\text{planned}}$. **Driving Score (DS)** weights RC by severe safety infractions (where $p_i \in (0, 1)$ is the penalty for infraction i occurring C_i times):

$$\text{DS} = \text{RC} \times \prod_i p_i^{C_i} \quad (23)$$

Minimum Time-to-Collision (minTTC) captures the strict safety margin under adversarial scenarios:

$$\text{minTTC} = \min_{t \in [0, T]} \left(\frac{\|x_{\text{ego}}(t) - x_{\text{agent}}(t)\|_2}{v_{\text{rel}}(t)} \right) \quad (24)$$

C.3 Background Agent Behavioral Modeling

Unlike traditional simulators that rely on deterministic open-loop log replay, evaluating E2E policies requires background agents that exhibit realistic, reactive, and physically plausible behaviors. We empower the background traffic with a coupled longitudinal-lateral behavioral model driven by the IDM and the MOBIL model.

Longitudinal Control (IDM). Background agents update their longitudinal acceleration a_{IDM} based on current velocity v , leader velocity v_{lead} , and the gap s :

$$a_{\text{IDM}}(s, v, \Delta v) = a_{\text{max}} \left[1 - \left(\frac{v}{v_0} \right)^\delta - \left(\frac{s^*(v, \Delta v)}{s} \right)^2 \right] \quad (25)$$

where $\Delta v = v - v_{\text{lead}}$, v_0 is desired velocity, and a_{max} is maximum acceleration. The desired minimum gap s^* is computed as $s^* = s_0 + vT + \frac{v\Delta v}{2\sqrt{a_{\text{max}}b_{\text{comf}}}}$, detailed in Algorithm 3.

Algorithm 4 MOBIL Lane-Changing Decision

Require: Ego vehicle C , Current lane L_{curr} , Neighboring lanes $\{L_{\text{target}}\}$
Ensure: Best lane choice L_{best}

- 1: $L_{\text{best}} \leftarrow L_{\text{curr}}, G_{\text{max}} \leftarrow -999.0$
- 2: Calculate current acceleration $a_c \leftarrow \text{IDM}(C, \text{Leader}_{\text{curr}})$
- 3: **for** each feasible L_{target} **do**
- 4: Identify New Leader ($\tilde{L}$) and New Follower ($\tilde{F}$) in L_{target}
- 5: Calculate $\tilde{a}_n \leftarrow \text{IDM}(\tilde{F}, C)$
- 6: **if** $\tilde{a}_n < -b_{\text{safe}}$ **then**
- 7: **continue** {Safety criterion violated, abort lane change}
- 8: **end if**
- 9: Calculate predicted ego acceleration $\tilde{a}_c \leftarrow \text{IDM}(C, \tilde{L})$
- 10: Calculate gain $G \leftarrow \tilde{a}_c - a_c$ {Simplified self-interest gain formulation}
- 11: **if** $G > a_{\text{thr}}$ **and** $G > G_{\text{max}}$ **then**
- 12: $G_{\text{max}} \leftarrow G, L_{\text{best}} \leftarrow L_{\text{target}}$
- 13: **end if**
- 14: **end for**
- 15: **return** L_{best}

Lateral Control (MOBIL). A lane change is executed if it satisfies a safety criterion (braking induced on the new follower $\tilde{a}_n \geq -b_{\text{safe}}$) and an incentive criterion:

$$\underbrace{(\tilde{a}_c - a_c)}_{\text{ego gain}} + p \underbrace{((\tilde{a}_n - a_n) + (\tilde{a}_o - a_o))}_{\text{neighbors' gain}} > a_{\text{thr}} \quad (26)$$

Algorithm 4 outlines this decision logic for complex urban maneuvers.

C.4 Interactive Closed-loop System Integration

The core value of DecoupleGS lies in bridging photorealistic rendering with fully interactive simulation. Unlike open-loop testing, our framework creates a dynamic game-theoretic environment where the ego-vehicle (driven by the E2E policy) and background agents (driven by IDM-MOBIL) react to one another continuously.

The closed-loop evaluation follows a synchronous step-by-step protocol. At each timestamp t , the simulator renders the multi-view sensor data based on the ego-vehicle’s current 6-DoF pose. The E2E planner consumes these images to infer the surrounding environment and outputs a local planned trajectory for the ego-vehicle. Concurrently, the background agent engine perceives the ego-vehicle’s current state and calculates the optimal longitudinal and lateral responses for all traffic participants. Finally, a global kinematic update resolves all trajectories to prevent physical overlap, advancing the entire scene to $t + 1$. This fully interactive data flow is summarized in Algorithm 5.

Algorithm 5 Interactive Closed-Loop E2E Evaluation Pipeline

Require: E2E Policy π_{ego} , Background Engine π_{bg} (IDM+MOBIL), Static Background $\mathcal{S}_{\text{bg}}$, Dynamic Assets $\{\mathcal{V}_k\}$, Simulation Time T_{max} , Step Δt

Ensure: Episode Driving Score (DS) and Minimum Time-to-Collision (minTTC)

- 1: Initialize $t \leftarrow 0$, Ego state $S_{\text{ego}}^{(0)}$, Background states $\mathbb{S}_{\text{bg}}^{(0)}$
- 2: **while** $t < T_{\text{max}}$ **and** No Terminal Collision **do**
- 3: // **1. High-Fidelity Sensor Simulation**
- 4: Render surround-view images $I_t \leftarrow \text{DecoupleGS}(S_{\text{ego}}^{(t)}, \mathbb{S}_{\text{bg}}^{(t)}, \mathcal{S}_{\text{bg}}, \{\mathcal{V}_k\})$
- 5: // **2. Ego-Vehicle Inference**
- 6: Predict ego trajectory $\mathcal{T}_{\text{ego}} \leftarrow \pi_{\text{ego}}(I_t, \text{Command})$
- 7: Compute next ego state $S_{\text{ego}}^{(t+1)}$ via kinematic bicycle model tracking $\mathcal{T}_{\text{ego}}$
- 8: // **3. Background Agents Interactive Response**
- 9: **for** each background agent $k \in \mathbb{S}_{\text{bg}}^{(t)}$ **do**
- 10: Identify surrounding leaders/followers including the Ego vehicle $S_{\text{ego}}^{(t)}$
- 11: Target lane $L_{\text{target}} \leftarrow \text{MOBIL}(\text{Agent}_k)$
- 12: Target acceleration $a_k \leftarrow \text{IDM}(\text{Agent}_k, \text{Leader}_{L_{\text{target}}})$
- 13: Compute next agent state $S_k^{(t+1)}$ via kinematic physical update
- 14: **end for**
- 15: // **4. Global State Synchronization & Metrics**
- 16: Synchronize global scene: update all 3DGS local-to-world transforms $\mathcal{T}(t+1)$
- 17: Calculate minTTC _{t} and update Route Completion (RC)
- 18: $t \leftarrow t + \Delta t$
- 19: **end while**
- 20: **return** Calculated DS and minTTC over the episode

D Additional Qualitative Results: Scene Decoupling and Editing

In this final section, we present extended qualitative results that showcase the versatility of our DecoupleGS framework in scene manipulation. By fundamentally decoupling the static background from dynamic agents, our method natively supports advanced editing capabilities without requiring any retraining or per-scene network optimization.

D.1 Scene Decoupling in Surround-View

The core advantage of our object-centric canonical representation is the ability to strictly isolate dynamic traffic participants from the persistent background across all Bird's-Eye-View (BEV) and sensor views.

As demonstrated in Figure 10, our framework successfully decomposes a complex driving scene. The top row displays the original unedited reconstruction across the standard six-camera surround-view suite. The middle row isolates the static 3DGS background $\mathcal{S}_{\text{bg}}$, cleanly erasing all dynamic entities without leaving "floaters" or shadow entanglement artifacts typically seen in holistic neural representations. The bottom row visualizes the extracted dynamic canonical assets



Fig.10: Surround-View Scene Decoupling. Top: Original reconstructed scene. Middle: Isolated static background with all dynamic agents perfectly removed. Bottom: Extracted dynamic foreground agents rendered independently. The decoupling is strictly consistent across the entire six-camera suite.

$\mathcal{V}_k$ rendered independently. Crucially, this decoupling maintains strict multi-view consistency and preserves the high-fidelity structural details of both the background infrastructure and the dynamic vehicles.

D.2 Interactive Scene Editing

Building upon the clean decoupling of the environment, DecoupleGS functions as a robust scenario generation engine that enables interactive and photorealistic scene editing.

Figure 11 illustrates this capability using the front-camera view. The original scene (top) contains the natively captured traffic flow. In the edited scene (bottom), we manipulate the dynamic assets by altering their spatial positions and substituting their appearances. Thanks to our map-guided registration and



Fig.11: Interactive Scene Editing. Top: Original front-camera view containing native traffic. Bottom: Edited scene where vehicle appearances and spatial positions have been explicitly manipulated. The modified assets seamlessly integrate with the background’s illumination and ground geometry.

proxy-based relighting modules, the manipulated and newly inserted vehicles naturally inherit the geometric grounding and photometric illumination of the environment, achieving seamless integration. This confirms the framework’s immense potential for generating diverse, adversarial closed-loop testing scenarios for E2E autonomous driving evaluation.